Plus longue sous-chaîne commune (naïf) + recherche KMP

LCS naïf | Recherche de motif de Knuth-Morris-Pratt

longest_common_naive.cpp | kmp_search.cpp | vérifié par exécution, `g++ -O2 -Wall`

1. Objectif

Étant données deux chaînes, résoudre deux problèmes liés :

1. **Plus longue sous-chaîne commune** : trouver le plus long bloc contigu partagé par les chaînes `a` et `b`.
2. **Recherche de motif (KMP)** : trouver toutes les occurrences d'un motif `pat` dans un texte `txt`, chevauchements inclus, en temps linéaire.

Résultats de référence (les deux programmes ont été exécutés localement) :

- `"abcdef"` vs `"zabcde"` → `"abcde"` (len 5)
- `"computer"` vs `"commuter"` → `"uter"` (len 4)
- `"ABABDABACDABABCABAB"` recherche `"ABABCABAB"` → 1 occurrence à l'indice 10
- `"aaaaa"` recherche `"aaa"` → occurrences en 0, 1, 2 (chevauchements inclus)

2. Plus longue sous-chaîne commune (méthode naïve)

Signature

```
LCSResult longestCommonSubstringNaive(
    const string &a, const string &b)
```

Retourne le plus long bloc partagé avec sa longueur et ses positions de départ dans `a` et `b`. Entrée vide → résultat vide.

Idée en 3 points

1. Essayer chaque paire de départ `(i, j)`.
2. Étendre `k` tant que `a[i+k] == b[j+k]`.
3. Garder le meilleur triplet vu jusqu'ici.

Aucun prétraitement, aucune mémoire supplémentaire.

Fonction testée telle quelle depuis `longest_common_naive.cpp:15`.

```
LCSResult longestCommonSubstringNaive(const string &a, const string &b) {
    LCSResult best;
    for (size_t i = 0; i < a.size(); i++) {
        for (size_t j = 0; j < b.size(); j++) {
            size_t k = 0;
            while (i + k < a.size() && j + k < b.size() && a[i + k] == b[j + k])
                k++;
            if (k > best.len) {
                best.len = k;
                best.posA = i;
                best.posB = j;
                best.value = a.substr(i, k);
            }
        }
    }
    return best;
}
```

Exemple détaillé : `a = "computer"`, `b = "commuter"`. Paires de départ sélectionnées :

(i, j)	a[i] / b[j]	extension	len
(0, 0)	c / c	`com`, puis `p` vs `m` arrête	3
(3, 3)	p / m	pas de correspondance	0
(4, 4)	u / u	`uter`, meilleur	4
(5, 5)	t / t	`ter`	3
(7, 7)	r / r	`r`	1

Lecture : la paire (0, 0) construit `"com"` (len 3) ; la paire (4, 4) construit `"uter"` (len 4) et devient la meilleure. Les autres paires ne la battent jamais, donc la réponse est `"uter"`.

Complexité : double boucle sur `n fois m` paires, chaque extension jusqu'à `L` → temps $O(n \cdot m \cdot L)$, mémoire supplémentaire `O(1)`. Les chaînes aléatoires coïncident rarement donc les extensions restent courtes ; les entrées en `"a"` répété sont le pire cas (section 3, tableau B).

Sorties de démonstration vérifiées (`longest_common_naive.cpp:33`):

Entrée (a / b)	Sortie	Statut
`abcdef` / `zabcde`	`"abcde"` (len 5)	OK
`computer` / `commuter`	`"uter"` (len 4)	OK
`hello` / `world`	`"l"` (len 1)	OK
`abc` / `xyz`	`""` (len 0)	OK
`""` / `abc`	`""` (len 0)	OK

3. Recherche de motif de Knuth-Morris-Pratt

Signatures

```
vector<int> computeLPS(const string &pat)
vector<int> kmpSearch(
    const string &txt, const string &pat)
```

`computeLPS` construit la table des préfixes ; `kmpSearch` retourne chaque indice de départ, chevauchements inclus.

Idée clé

En cas de différence, ne pas reculer le pointeur du texte. Replier avec `j = lps[j-1]` et reprendre la comparaison.

L'indice du texte `i` ne recule jamais, donc le travail total reste linéaire.

Fonction de préfixe telle quelle depuis `kmp_search.cpp:10`.

```
vector<int> computeLPS(const string &pat) {
    vector<int> lps(pat.size(), 0);
    int len = 0; // length of current longest prefix-suffix
    for (size_t i = 1; i < pat.size();) {
        if (pat[i] == pat[len]) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1]; // fall back, do not advance i
        } else {
            lps[i++] = 0;
        }
    }
    return lps;
}
```

Recherche telle quelle depuis `kmp_search.cpp:27`.

```
vector<int> kmpSearch(const string &txt, const string &pat) {
    vector<int> occ;
    if (pat.empty())
        return occ;
    vector<int> lps = computeLPS(pat);
    size_t i = 0; // index in txt
    size_t j = 0; // index in pat
    while (i < txt.size()) {
        if (txt[i] == pat[j]) {
            i++;
            j++;
            if (j == pat.size()) {
                occ.push_back((int)(i - j));
                j = lps[j - 1]; // allow overlapping matches
            }
        } else if (j > 0) {
            j = lps[j - 1]; // skip re-comparisons, key KMP idea
        } else {
            i++;
        }
    }
    return occ;
}
```

Table LPS pour `pat = "ABABCABAB"` (sortie mesurée `[0,0,1,2,0,1,2,3,4]`) :

i	pat[i]	lps[i]	sens
0	A	0	un seul caractère, pas de bordure propre
1	B	0	`"AB"` : pas de bordure
2	A	1	`"ABA"` : bordure `"A"`
3	B	2	`"ABAB"` : bordure `"AB"`
4	C	0	rupture, repli à 0
5	A	1	bordure `"A"` à nouveau
6	B	2	bordure `"AB"`
7	A	3	bordure `"ABA"`
8	B	4	bordure `"ABAB"`

Trace de recherche (événements clés) : `txt = "ABABDABACDABABCABAB"`, `pat = "ABABCABAB"`, occurrence en 10.

étape	pos. texte	pos. motif	événement
1	`i = 0..3`	`j = 0..3`	`ABAB` correspond, puis `txt[4] = D` vs `pat[4] = C` : repli `j = lps[3] = 2`
2	`i = 5..9`	`j = 2..`	le deuxième essai échoue vers `txt[8] = C` ; KMP décale via LPS au lieu de reculer `i`
3	`i = 10..18`	`j = 0..8`	correspondance complète de longueur 9 → occurrence en 10
4	fin	—	1 occurrence en {10} ; les motifs à chevauchement repartiraient de `j = lps[8] = 4`

Vérification du chevauchement : `txt = "aaaaa"`, `pat = "aaa"` → occurrences en 0, 1, 2, car après chaque occurrence `j` repart à `lps[2] = 2` au lieu de 0.

Complexité : LPS coûte temps `O(m)` et espace `O(m)` ; la recherche coûte temps `O(n)`. Total temps `O(n + m)`, espace `O(m)`. L'indice du texte avance au plus `n` fois et l'indice du motif recule au plus `n` fois, donc les comparaisons sont bornées par `2n`.

4. Mesures de performance

Tous les chiffres ont été mesurés localement avec `g++ -O2`. Mesures suivies : temps écoulé, nombre de comparaisons de caractères, mémoire supplémentaire et comportement dans le pire cas.

Mesures en bref : le temps montre l'échelle ; le nombre de comparaisons l'explique ; la mémoire est `O(1)` supplémentaire pour le LCS naïf contre `O(m)` pour la table LPS ; le pire cas est cubique pour le LCS naïf sur caractères répétés mais d'au plus `2n` comparaisons pour la recherche KMP.

Tableau A — LCS naïf, chaînes aléatoires partageant un bloc de 20 caractères (cas moyen) :

n	temps	comparaisons
100	0.07 ms	13k
500	1.42 ms	262k
1000	5.71 ms	1.04M
2000	22.6 ms	4.16M

Lecture : le temps est multiplié par 4 environ quand `n` double (0.07 $\rightarrow$ 1.42 $\rightarrow$ 5.71 $\rightarrow$ 22.6 ms) et les comparaisons suivent n^2, la courbe quadratique attendue pour une entrée aléatoire.

Tableau B — LCS naïf, pire cas, `"a" * n` contre `"a" * n` :

n	temps	comparaisons
200	7.8 ms	2.7M
500	118 ms	41.8M
1000	577 ms	334M

Lecture : chaque paire correspond sur une longue plage, donc la boucle d'extension explose : 2.7M $\rightarrow$ 41.8M $\rightarrow$ 334M comparaisons. Cette explosion cubique explique pourquoi la méthode naïve ne convient qu'aux petites entrées.

Tableau C — recherche KMP contre recherche naïve :

cas	naïve	KMP
adverse, `pat = "aaab"`, `txt = "a" * 50000 + "b"`	199992 cmp / 0.38 ms	99998 cmp / 0.18 ms
classique, occurrence en {10}	29 cmp	23 cmp
échelle n = 20k	49k cmp / 0.12 ms	26k cmp / 0.10 ms
échelle n = 100k	247k cmp / 0.43 ms	132k cmp / 0.31 ms

Lecture : sur l'entrée adverse, le balayage naïf revérifie chaque décalage (199992 contre 99998 comparaisons, 0.38 contre 0.18 ms) car il recule après chaque quasi-correspondance tandis que KMP ne recule jamais. À grande échelle, KMP utilise environ 1,9x moins de comparaisons (49k contre 26k ; 247k contre 132k) et les deux passent à l'échelle linéairement, ce qui confirme le comportement en `O(n + m)`.

5. Conclusion et sources

Le LCS naïf est court, évidemment correct et adapté aux petites chaînes, mais son temps en $O(n \cdot m \cdot L)$ devient cubique sur entrée répétitive (577 ms pour `n = 1000`). KMP paie une mémoire en `O(m)` pour la table LPS et obtient une recherche linéaire avec au plus `2n` comparaisons, divisant environ par deux le travail du balayage naïf sur les entrées adverses et volumineuses. Règle pratique : naïf pour la clarté et les minuscules entrées, KMP dès que le texte est grand ou que le motif se chevauche lui-même.

Sources :

1. Nayuki — Recherche de motifs de Knuth-Morris-Pratt. https://www.nayuki.io/page/knuth-morris-pratt-string-matching
2. GeeksforGeeks — plus longue sous-chaîne commune (article sur la programmation dynamique).
3. Wikipédia — Problème de la plus longue sous-chaîne commune.